Unified Modeling Language (UML)

1. What is UML?

UML (Unified Modeling Language) is a standard way to visually design and represent software systems using diagrams.

The main purpose is to have a common visual language that shows how classes, objects, relationships, and interactions work in a system.

This makes it easier for developers and teams to understand, discuss, and plan the system before writing actual code.

2. Why UML Matters

Visualization: UML diagrams let you see the system visually, like a blueprint — making it easy to understand how parts are structured and connected.

Documentation: UML diagrams act as detailed documentation of your software design, very helpful for maintenance and scaling.

Communication: UML creates a common language that designers, developers, and teams can all understand and discuss without confusion.

Standardization: Since UML is a universally accepted standard, everyone in the software industry reads these diagrams the same way, no matter where you work.

3. Basic Elements

3.1 Class

A blueprint or template for creating objects. It has a name, properties (attributes), and actions (methods) that objects can perform.

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age  = age

    def login(self):
        pass
```

3.2 Interface

In Python, we use Abstract Classes to define which methods a class must have. It tells what a class should do, not how to do it.

```
from abc import ABC, abstractmethod

class Loginable(ABC):
    @abstractmethod
    def login(self):
        pass
```

3.3 Other Key Elements

Object: A real working copy of a class created when the program runs. Example: user1 = User("Anirudh", 25)

Association: A connection between two classes showing how their objects interact or relate to each other.

Inheritance: An "is-a" relationship where a child class inherits properties and methods from a parent class. Example: class Dog(Animal)

Composition: A strong "has-a" relationship — the contained object cannot exist without the container. Example: Car has Engine

Aggregation: A weak "has-a" relationship — the contained object can exist independently. Example: Classroom has Students

4. Class Diagrams

A class diagram shows the blueprint of your system — classes, their attributes, methods, and how they connect to each other.

Student
-name : str -roll_no : int -cgpa : float
+attend_class() +submit_assignment()

```
class Student:
    def __init__(self, name, roll_no, cgpa):
        self.__name    = name       # private attribute
        self.__roll_no = roll_no    # private attribute
        self.__cgpa    = cgpa       # private attribute

    def attend_class(self):         # public method
```

```
        print(f"{self.__name} is attending class.")

    def submit_assignment(self):    # public method
        print(f"{self.__name} submitted the assignment.")
```

5. Association

Association is a connection between two or more classes that shows how their objects are related. When one class's object uses or interacts with another class's object, that's association.

Simple terms: "Class A knows about Class B and can use it"


```
class Student:
    def __init__(self, name):
        self.name = name
    def get_name(self):
        return self.name

class Teacher:
    def __init__(self, name):
        self.name = name
    def teach(self, student: Student):   # Association
        print(f"{self.name} is teaching {student.get_name()}")

# Usage
s = Student("Anirudh")
t = Teacher("Prof. Sharma")
t.teach(s)   # Output: Prof. Sharma is teaching Anirudh
```

6. Aggregation

Aggregation is a weak "has-a" relationship where one class contains objects of another class, BUT the contained objects can survive independently even if the container is destroyed.

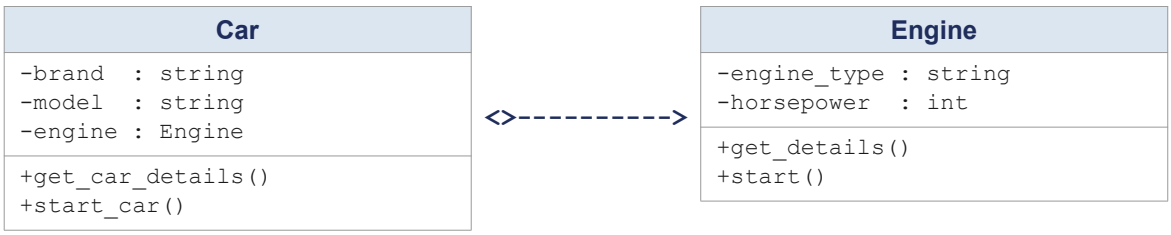
Simple terms: "Class A has Class B, but B can exist without A"

```
class Student:
    def __init__(self, name, roll_no):
        self.name    = name
        self.roll_no = roll_no

class Department:
    def __init__(self, dept_name):
        self.dept_name = dept_name
        self.students  = []        # Aggregation: holds Student objects
    def add_student(self, student):
        self.students.append(student)
    def show_students(self):
        for s in self.students:
            print(s.name, s.roll_no)

# Students exist independently — dept closure does NOT destroy them
s1 = Student("Anirudh", 101)
s2 = Student("Priya",   102)
dept = Department("Computer Science")
dept.add_student(s1)
dept.add_student(s2)
dept.show_students()
```

7. Composition

Composition is a strong "has-a" relationship where one class owns objects of another class. If the container object is destroyed, the contained objects are also destroyed automatically.

Simple terms: "Class A owns Class B completely — if A dies, B also dies"



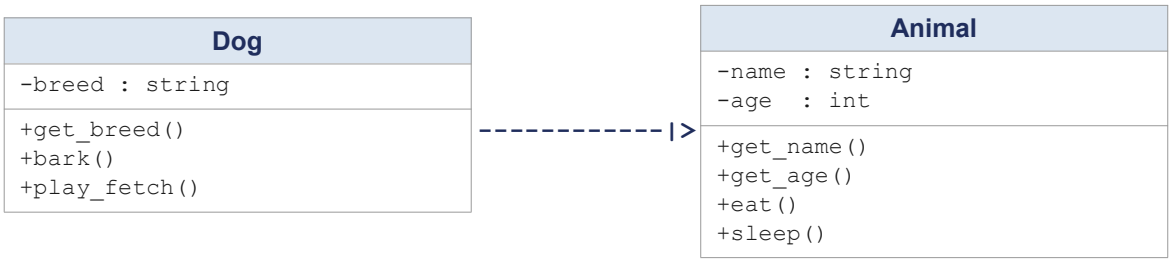

```
class Engine:
    def __init__(self, engine_type, horsepower):
        self.engine_type = engine_type
        self.horsepower  = horsepower
    def start(self):
        print(f"{self.engine_type} engine started!")

class Car:
    def __init__(self, brand, model, engine_type, horsepower):
        self.brand  = brand
```

```
        self.model  = model
        self.engine = Engine(engine_type, horsepower)  # Composition
    def start_car(self):
        self.engine.start()   # Car controls Engine lifecycle

# When my_car is deleted, its Engine is also gone
my_car = Car("Toyota", "Camry", "V6", 268)
my_car.start_car()   # Output: V6 engine started!
```

8. Inheritance

Inheritance defines an "is-a" relationship where a child class (subclass) inherits properties and behaviors from a parent class (superclass).

Simple terms: "Child class IS-A type of Parent class — child gets all features of Parent automatically"

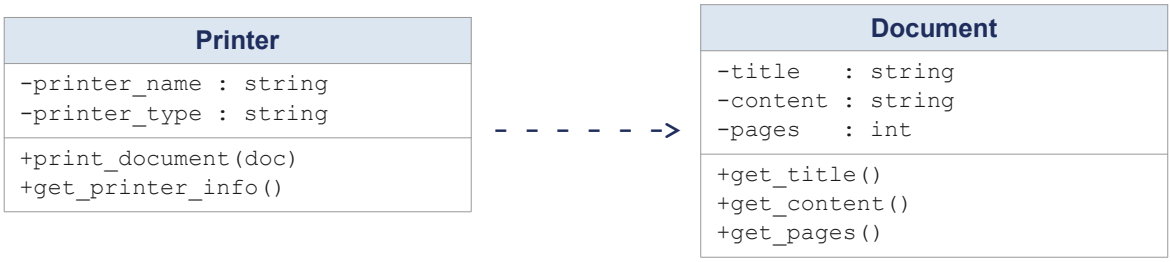



```
class Animal:
    def __init__(self, name, age):
        self.name = name
        self.age  = age
    def eat(self):   print(f"{self.name} is eating.")
    def sleep(self): print(f"{self.name} is sleeping.")

class Dog(Animal):               # Dog IS-A Animal
    def __init__(self, name, age, breed):
        super().__init__(name, age)   # Inherit from Animal
        self.breed = breed
    def bark(self):       print("Woof!")
    def play_fetch(self): print(f"{self.name} is playing fetch!")

d = Dog("Buddy", 3, "Labrador")
d.eat()        # Inherited  ->  Buddy is eating.
d.bark()       # Own method ->  Woof!
d.play_fetch() # Own method ->  Buddy is playing fetch!
```

9. Dependency

Dependency is a weak relationship where one class temporarily uses or relies on another class through method parameters, return types, or temporary usage — not permanent ownership.

Simple terms: "Class A uses Class B for a short time to do work, then forgets about it"




```
class Document:
    def __init__(self, title, content, pages):
        self.title   = title
        self.content = content
        self.pages   = pages
    def get_title(self): return self.title
    def get_pages(self): return self.pages

class Printer:
    def __init__(self, name, printer_type):
        self.printer_name = name
        self.printer_type = printer_type
    # Dependency: Document passed as parameter — not stored permanently
    def print_document(self, document: Document):
        print(f'Printing "{document.get_title()}"'
              f' ({document.get_pages()} pages) on {self.printer_name}')

doc = Document("Report", "Content here...", 10)
p   = Printer("HP LaserJet", "Laser")
p.print_document(doc)
# Output: Printing "Report" (10 pages) on HP LaserJet
```

10. Summary

Quick reference for all UML relationships:

Relationship	Type	Key Rule	Notation
Association	General	A knows B and can use it	`A --> B`
Aggregation	Weak has-a	B can exist without A	`A o--> B`
Composition	Strong has-a	B dies when A dies	`A <>--> B`
Inheritance	is-a	Child IS-A type of Parent	`Child --|> Parent`
Dependency	Temporary use	A uses B briefly via method parameter	`A - - -> B`
Realization	Contract	Class implements interface methods	`A - -|> Interface`